

Artificial Intelligence Concepts and Machine Learning Techniques (AI – 103)

Walaa H. Elashmawi
Professor of AI

 walaa.hassan@miuegypt.edu.eg

Unsupervised ML

Supervised learning vs. unsupervised learning

Supervised
learning:

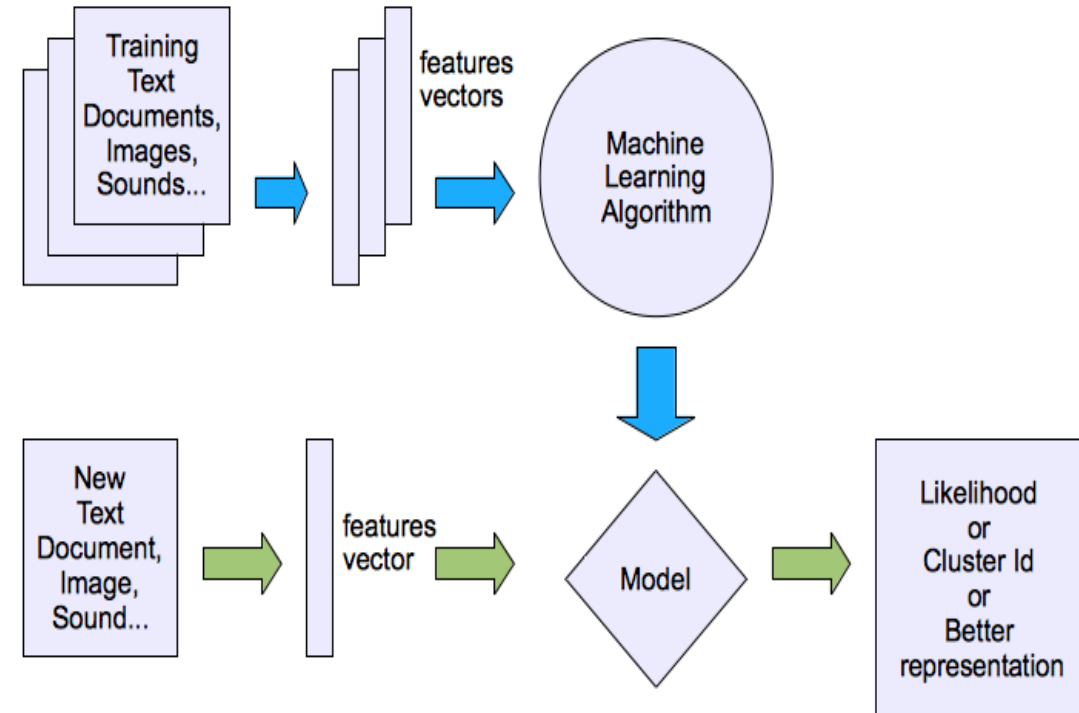
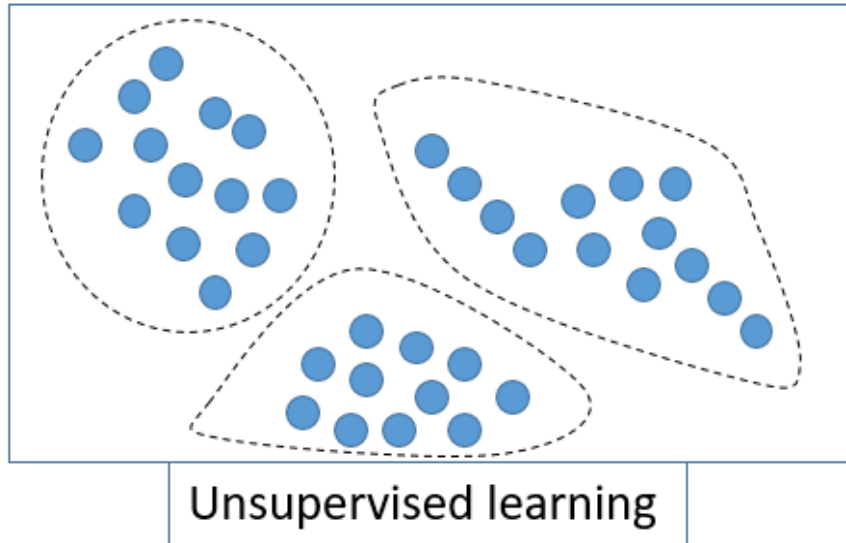
Discover patterns in the data that relate data attributes with a target (class) attribute.
These patterns are then utilized to predict the values of the target attribute in future data instances.

Unsupervised
learning:

The data have no target attribute.
We want to explore the data to find some intrinsic structures in them.

Unsupervised Learning

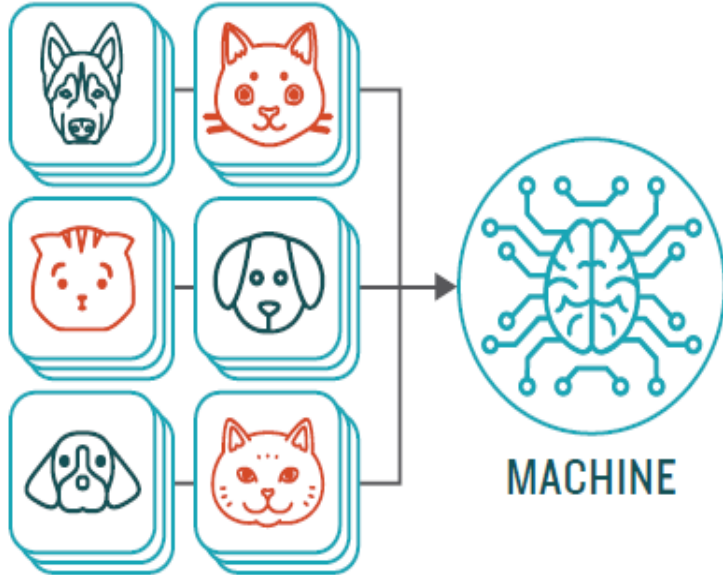
The correct classes of the training data are not known.



Unsupervised Learning

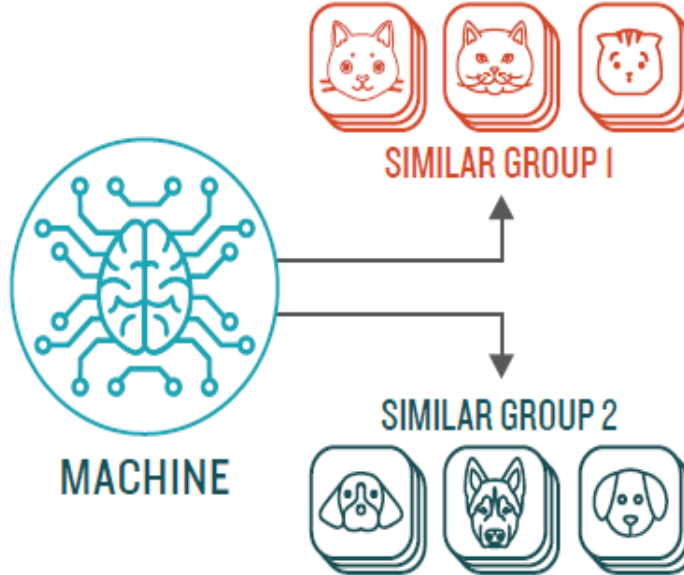
STEP 1

Provide the machine learning algorithm uncategorized, unlabeled input data to see what patterns it finds



STEP 2

Observe and learn from the patterns the machine identifies

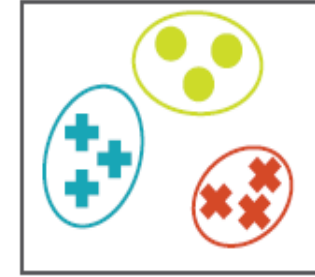


TYPES OF PROBLEMS TO WHICH IT'S SUITED

CLUSTERING

Identifying similarities in groups

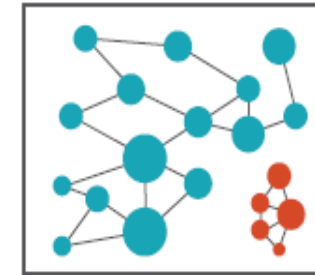
For Example: Are there patterns in the data to indicate certain patients will respond better to this treatment than others?



ANOMALY DETECTION

Identifying abnormalities in data

For Example: Is a hacker intruding in our network?





Types of Unsupervised Learning

Unsupervised Learning Algorithms

Clustering

- K-Means
- Polynomial
- Hierarchical
- Fuzzy C-Means

Dimensionality Reduction

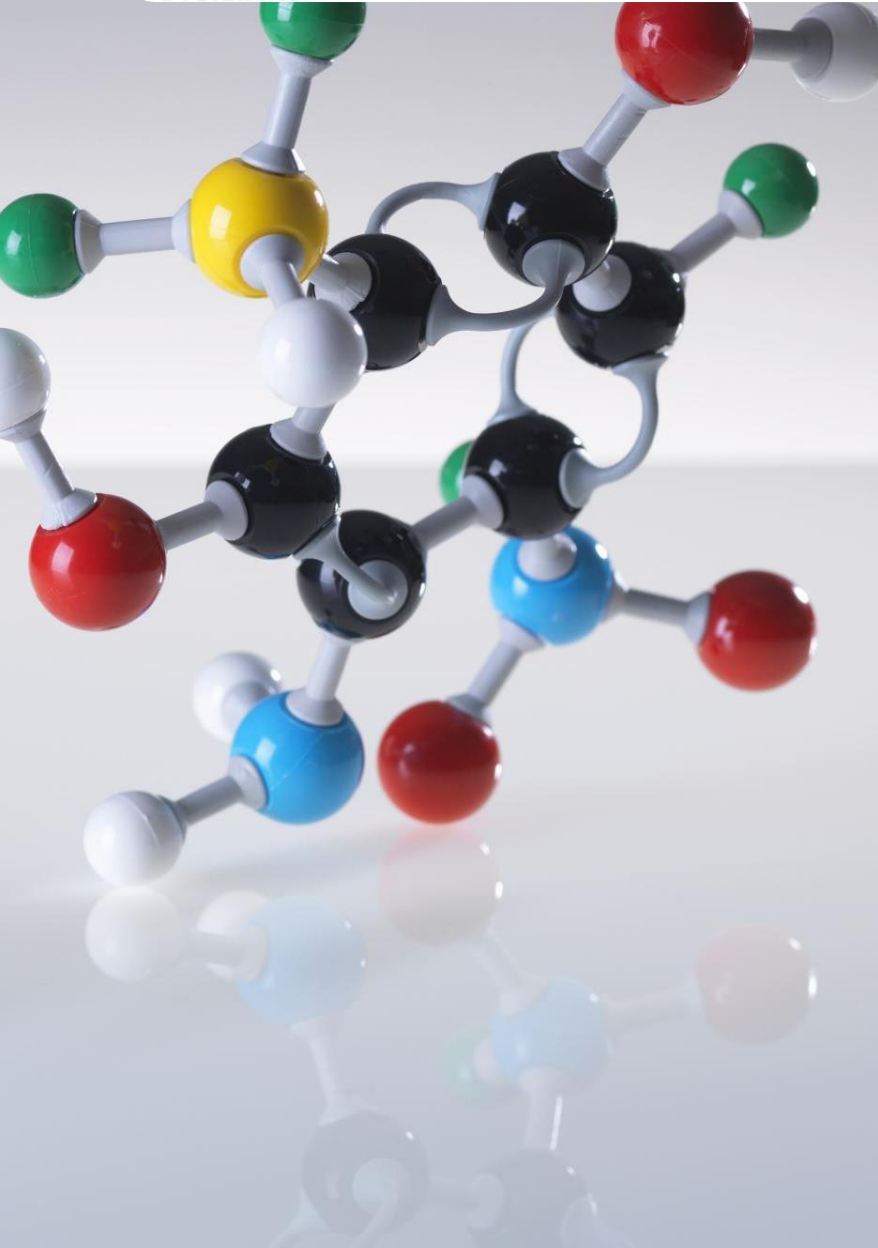
- Principal Component Analysis
- Kernel Principal Analysis

Association (Data Mining)

- Apriori Algorithm
- Eclat Algorithm
- FP-Growth Algorithm

Clustering

- **Clustering:** It is used to classify the objects into clusters on the basis of similarities.
- It classifies the objects into clusters through checking, which data points have less or more similarities.
- Commonalities between the data objects are found by this cluster analysis, and then these data-objects get categorized on the basis of presence and absence of those similarities.
- The most popularly used clustering techniques are **k-means** (divisive) and **hierarchical** (agglomerative).



Dimensionality Reduction

- **Dimensionality Reduction (DR)** is one of the methods of unsupervised machine learning in which we use to reduce the dimensions or input data variable in dataset to make the process smoother.
- It can also be understood as the techniques which is also used in classification and regression datasets to make a good predictive model.
- There are many more techniques in DR method, but the most-knowns are:
 1. Linear Discriminant Reduction (includes in Supervised Machine Learning)
 2. Principle Component Analysis (counted in Unsupervised Machine learning)

Association

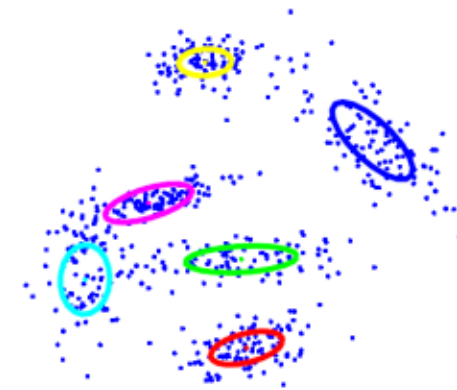
- An association rule is an unsupervised learning method which is used for find the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item.
- A typical example of Association rule is **Market Basket Analysis**.

Clustering

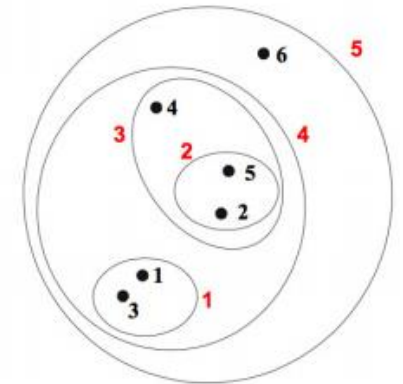
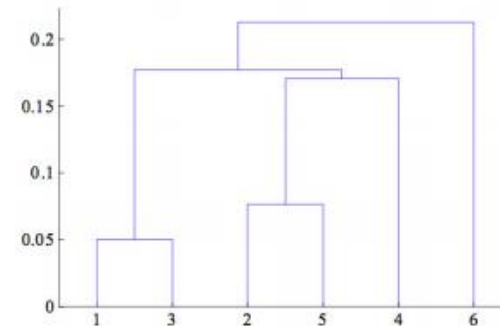
Approaches to Clustering

Broadly specified, there are two classes of clustering algorithms:

1. **Centroid approaches**: We guess the *centroid* (central point) in each cluster and assign points to the cluster of their nearest centroid.



2. **Hierarchical approaches**: We begin assuming that each point is a cluster by itself. We repeatedly merge nearby clusters, using some measure of how close two clusters are (e.g., distance between their centroids), or how good a cluster the resulting group would be (e.g., the average distance of points in the cluster from the resulting centroid.)





What is clustering for?

Let us see some real-life examples

- **Example 1:** groups people of similar sizes together to make “small”, “medium” and “large” T-Shirts.
 - Tailor-made for each person: too expensive
 - One-size-fits-all: does not fit all.
- **Example 2:** In marketing, segment customers according to their similarities
 - To do targeted marketing.
- **Example 3:** Given a collection of text documents, we want to organize them according to their content similarities,
 - To produce a topic hierarchy

What is clustering for? *(cont...)*

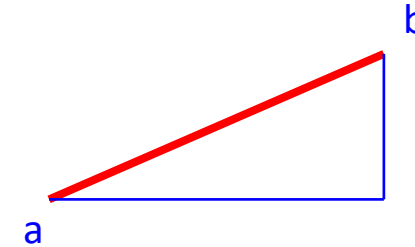
- In fact, clustering is one of the most utilized data mining techniques.
 - It has a long history, and used in almost every field, e.g., medicine, psychology, botany, sociology, biology, archeology, marketing, insurance, libraries, etc.
 - In recent years, due to the rapid increase of online documents, text clustering becomes important.



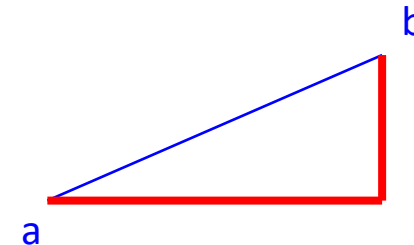
K-dimensional Euclidean space

The distance between any two points, say $a = [a_1, a_2, \dots, a_k]$ and $b = [b_1, b_2, \dots, b_k]$ is given in some manner such as:

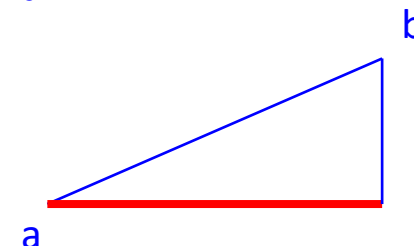
1. Common distance ("L₂ norm") : $\sqrt{\sum_{i=1}^k (a_i - b_i)^2}$



2. Manhattan distance ("L₁ norm"): $\sum_{i=1}^k |a_i - b_i|$



3. Max of dimensions ("L_∞ norm"): $\max_{i=1}^k |a_i - b_i|$



Non-Euclidean spaces

Here are some examples where a distance measure without a Euclidean space makes sense.

- Web pages: Roughly 10^8 -dimensional space where each dimension corresponds to one word. Rather use vectors to deal with only the words actually present in documents a and b.
- Character strings, such as DNA sequences: Rather use a metric based on the LCS---Lowest Common Subsequence.
- Objects represented as sets of symbolic, rather than numeric, features: Rather base similarity on the proportion of features that they have in common.



Non-Euclidean spaces (cont'd)

- $\text{object1} = \{\text{small, red, rubber, ball}\}$
- $\text{object2} = \{\text{small, blue, rubber, ball}\}$
- $\text{object3} = \{\text{large, black, wooden, ball}\}$

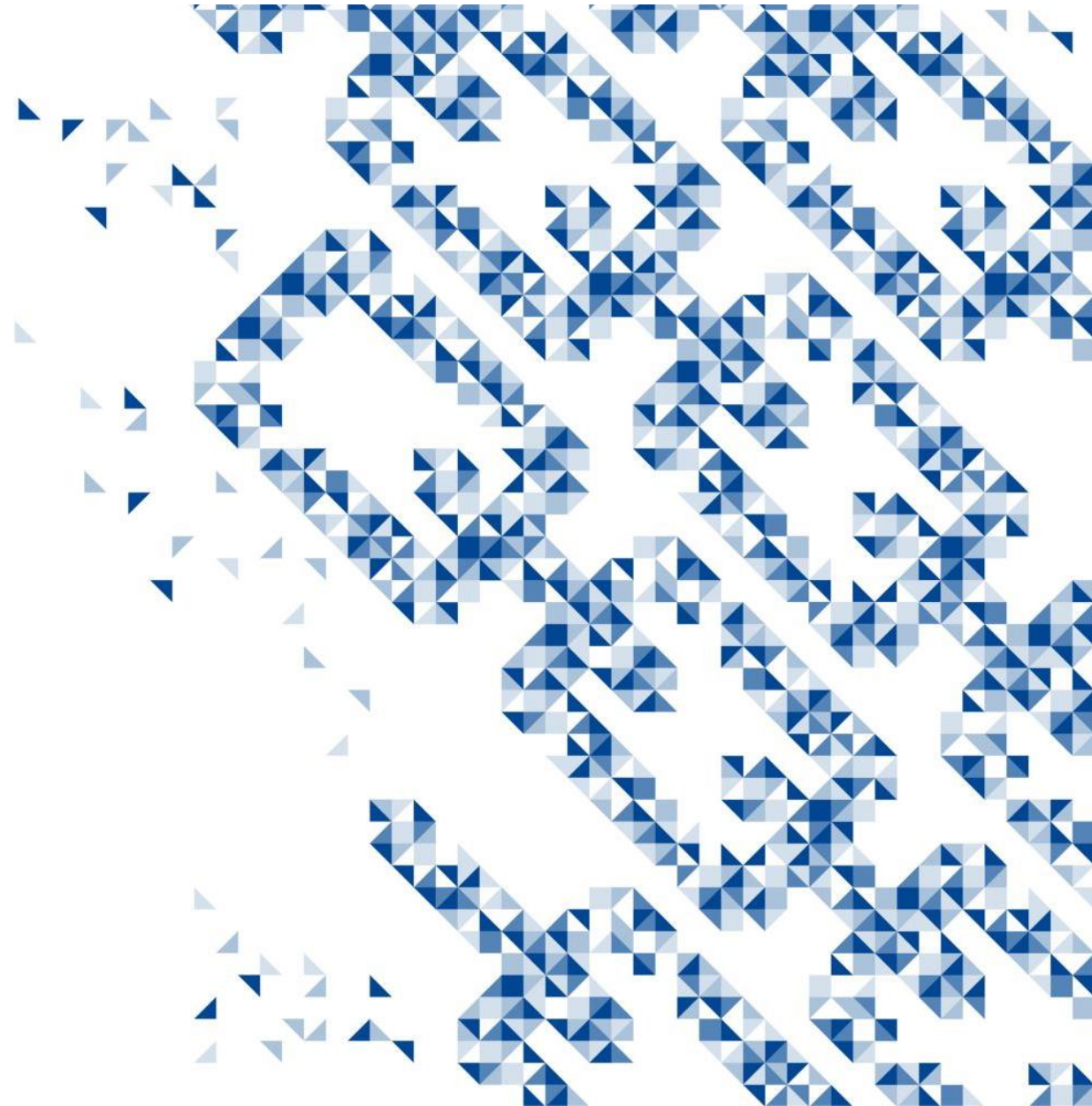
✓ $\text{similarity}(\text{object1}, \text{object2}) = 3 / 4$

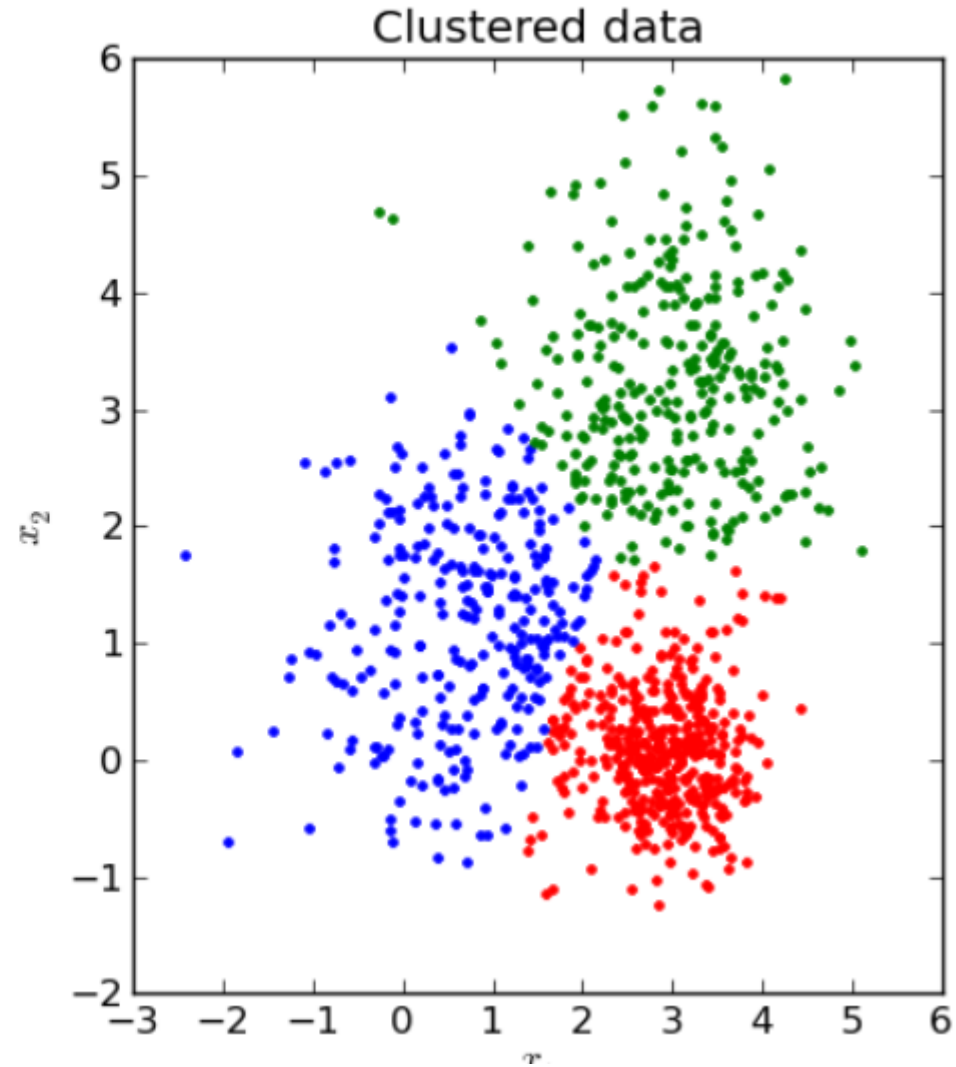
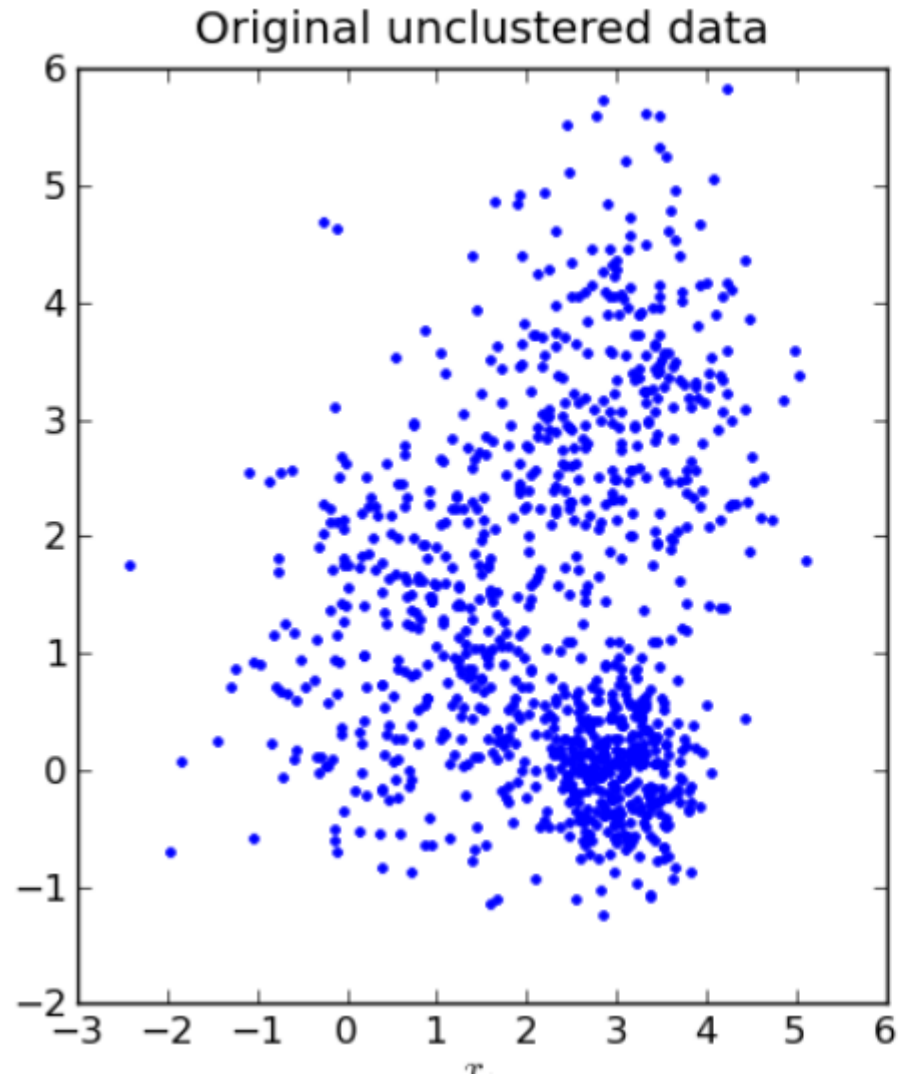
✓ $\text{similarity}(\text{object1}, \text{object3}) = \text{similarity}(\text{object2}, \text{object3}) = 1/4$

Note that it is possible to assign different weights to features.

k-means algorithm

- The key objective of a k-means algorithm is to organize data into clusters such that there is high intra-cluster similarity and low inter-cluster similarity. An item will only belong to one cluster, not several, that is, it generates a specific number of disjoint, nonhierarchical clusters.





The generalized algorithm

Assuming we have inputs $x_1, x_2, x_3, \dots, x_n$ and value of K

Step 1 - Pick K random points as cluster centers called centroids.

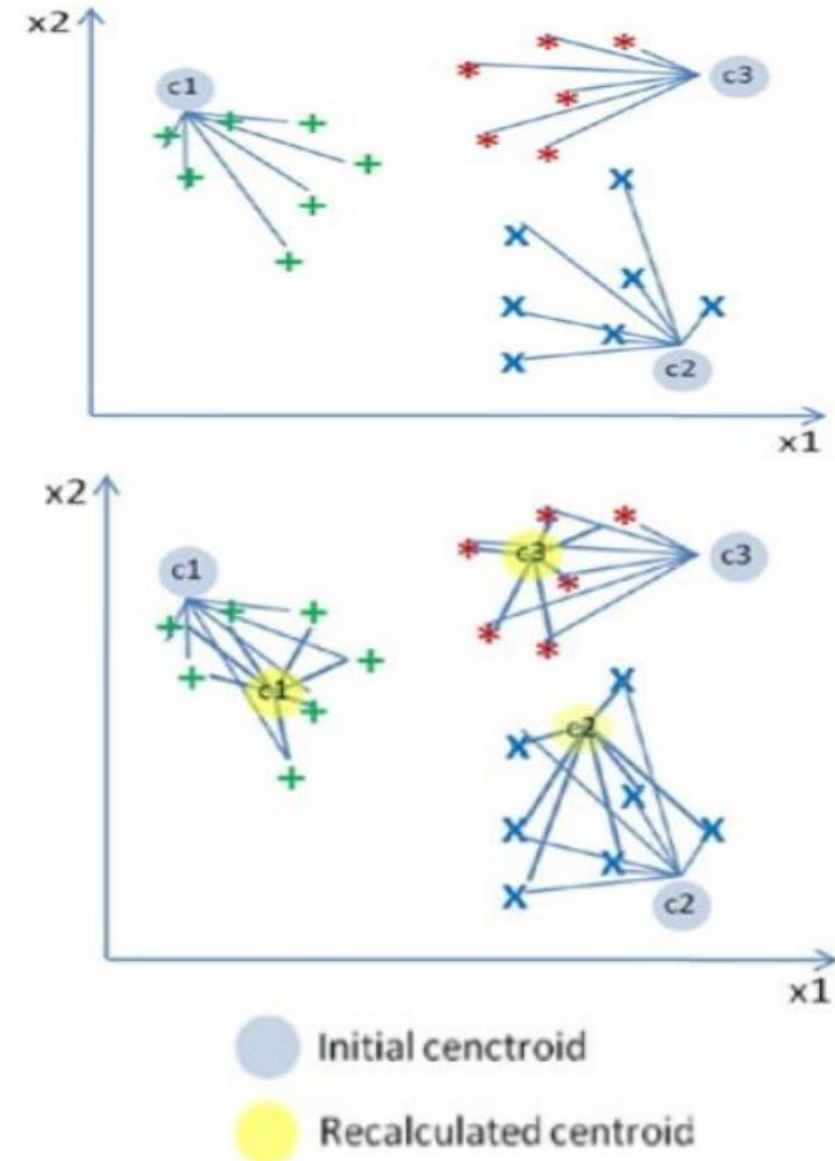
Step 2 - Assign each x_i to nearest cluster by calculating its distance to each centroid.

Step 3 - Find new cluster center by taking the average of the assigned points.

Step 4 - Repeat Step 2 and 3 until none of the cluster assignments change.

Practical Ex:

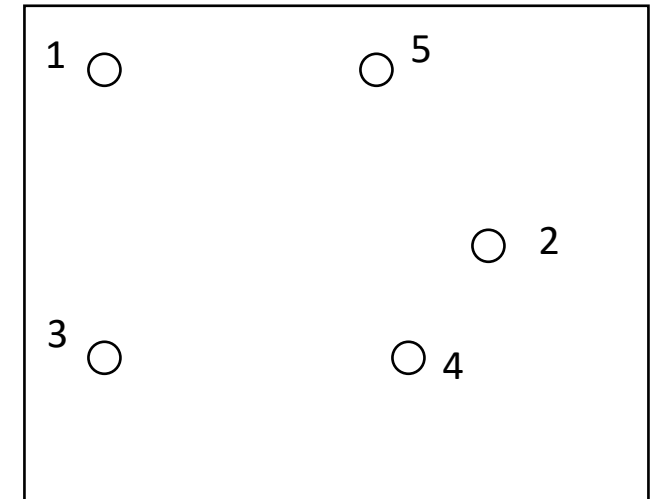
https://www.youtube.com/watch?v=oLp8OQKR_N0



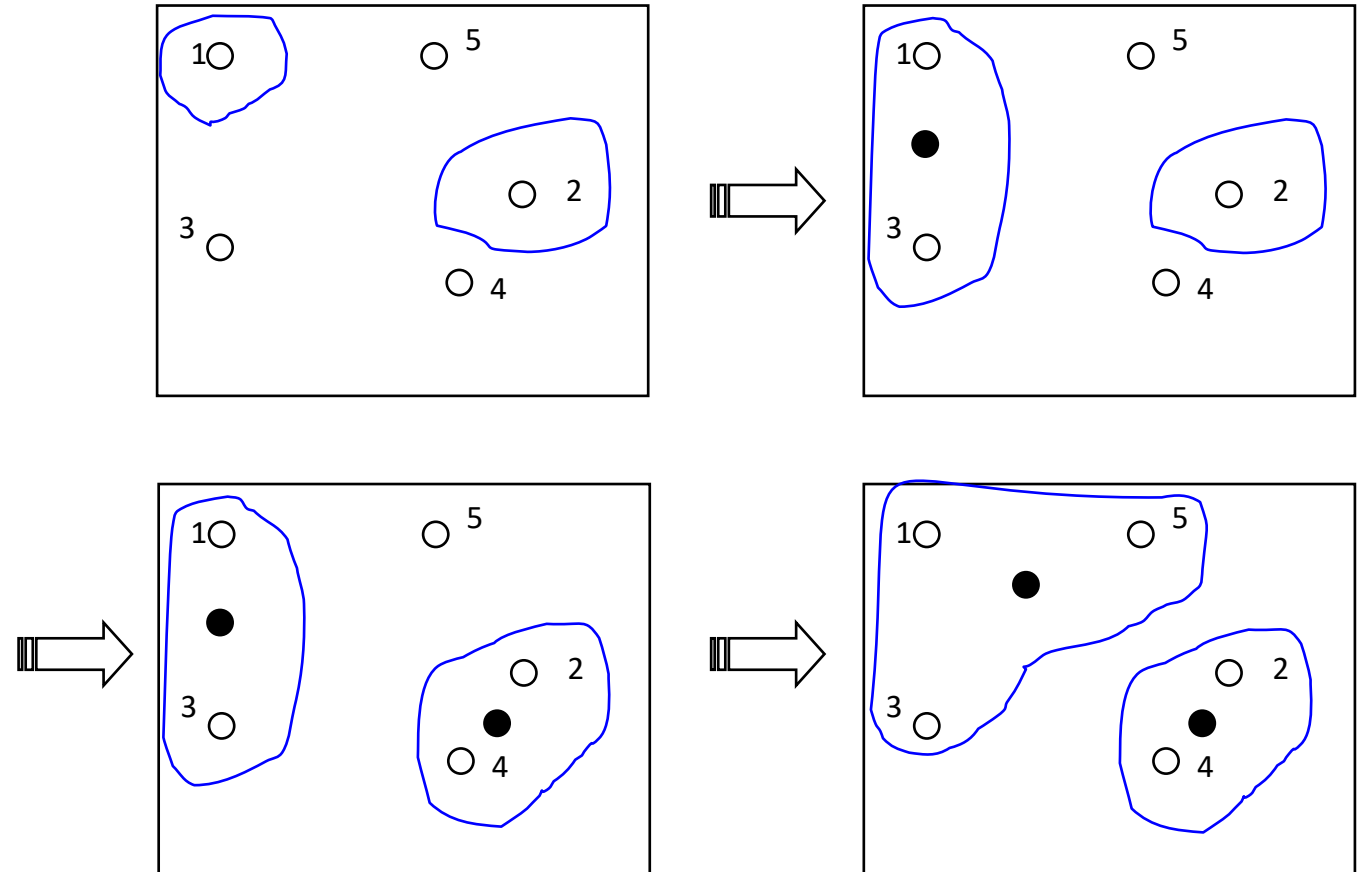
The k-means algorithm

- Pick k cluster centroids.
- Assign points to clusters by picking the closest centroid to the point in question. As points are assigned to clusters, the centroid of the cluster may migrate.

Example: Suppose that $k = 2$ and we assign points 1, 2, 3, 4, 5, in that order. Outline circles represent points, filled circles represent centroids.



The k-means algorithm example (cont'd)



Key issues: How to initialize the k centroids? And How to determine k ?

How many clusters?

There are two commonly used methods to determine the ideal number of clusters possible in K-means

– Elbow Method:

- It based on the sum of squared error (SSE) for some values of k (for example 2, 4, 6, 8, etc.)
- SSE measures the total distance between data points and their assigned cluster centroids
- As k increases, SSE decreases
- The optimal number of clusters is chosen at the “**elbow point**”, where the decrease in SSE becomes marginal

– Silhouette Method:

- Measures how well each data point fits within its assigned cluster
- Silhouette coefficient ranges from **-1 to +1**
- Close to **+1** → well-clustered
- Around **0** → overlapping clusters
- Negative values → possibly misclassified points
- The optimal number of clusters is the value of k that **maximizes the average silhouette score**

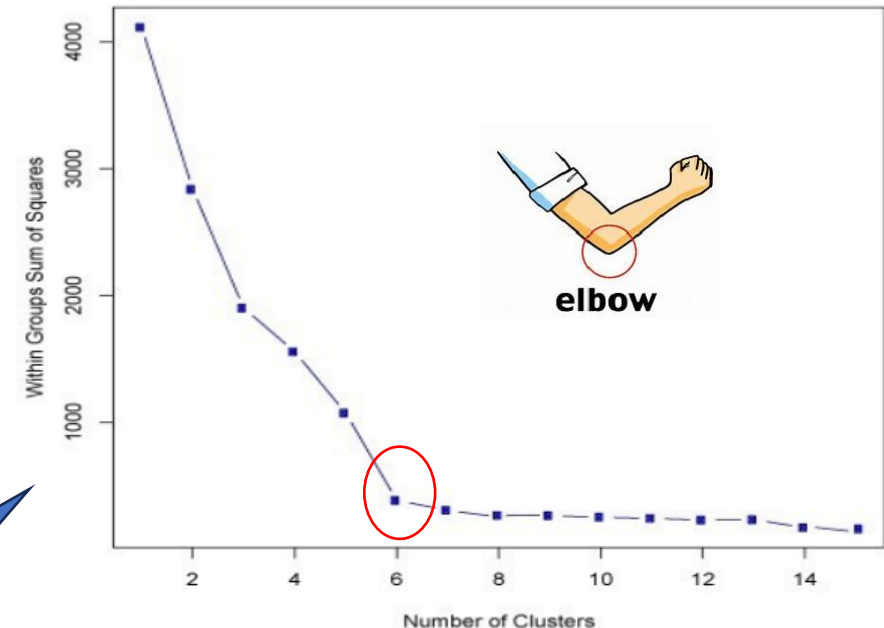
Elbow Method

- The SSE is defined as the sum of the squared distance between each member of the cluster and its centroid. Mathematically:

$$SSE = \sum_{i=1}^K \sum_{x \in c_i} dist(x, c_i)^2$$

- If you plot k against the SSE, you will see that the error decreases as k gets larger; this is because when the number of clusters increases, they should be smaller, so distortion is also smaller. The idea of the elbow method is to choose the k at which the SSE decreases abruptly. This produces an "elbow effect" in the graph

In this case, k=6 is the value that the Elbow method has selected



Silhouette Method

- It's a way to find out the optimum value for k during k-means clustering.
 - Silhouette values lies in the range of $[-1, 1]$. A value of +1 indicates that the sample is far away from its neighboring cluster and very close to the cluster its assigned.
 - Similarly, value of -1 indicates that the point is close to its neighboring cluster than to the cluster its assigned.
 - A value of 0 means its at the boundary of the distance between the two cluster. Value of +1 is idea and -1 is least preferred. Hence, higher the value better is the cluster configuration.
- In the data, lets define $a(i)$ to be the mean distance of point (i) w.r.t to all the other points in the cluster its assigned (A). We can interpret $a(i)$ as how well the point is assigned to the cluster. Smaller the value better the assignment.
 - Similarly, lets define $b(i)$ to be the mean distance of point(i) w.r.t. to other points to its closet neighboring cluster (B). The cluster (B) is the cluster to which point (i) is not assigned to but its distance is closest amongst all other cluster.



- Thus, the silhouette $s(i)$ can be calculated as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Measuring the quality of the clusters

A good clustering is one where:

(intra-cluster distance) the sum of distances between objects in the same cluster are minimized (high intra similarity).

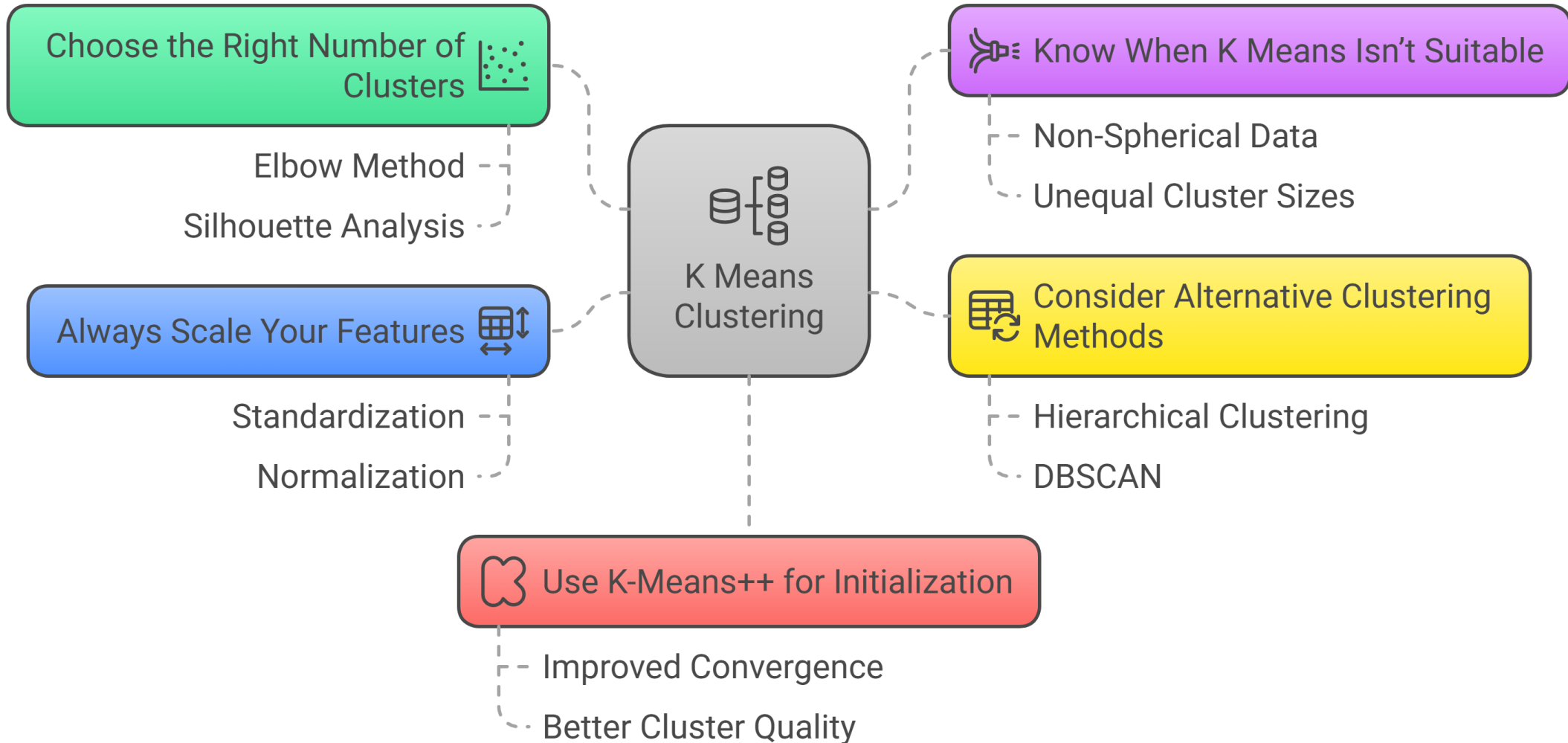
(inter-cluster distance) while the distances between different clusters are maximized (low inter similarity)..

The objective is to minimize $F(\text{intra}, \text{inter})$

K-means Clustering Applications

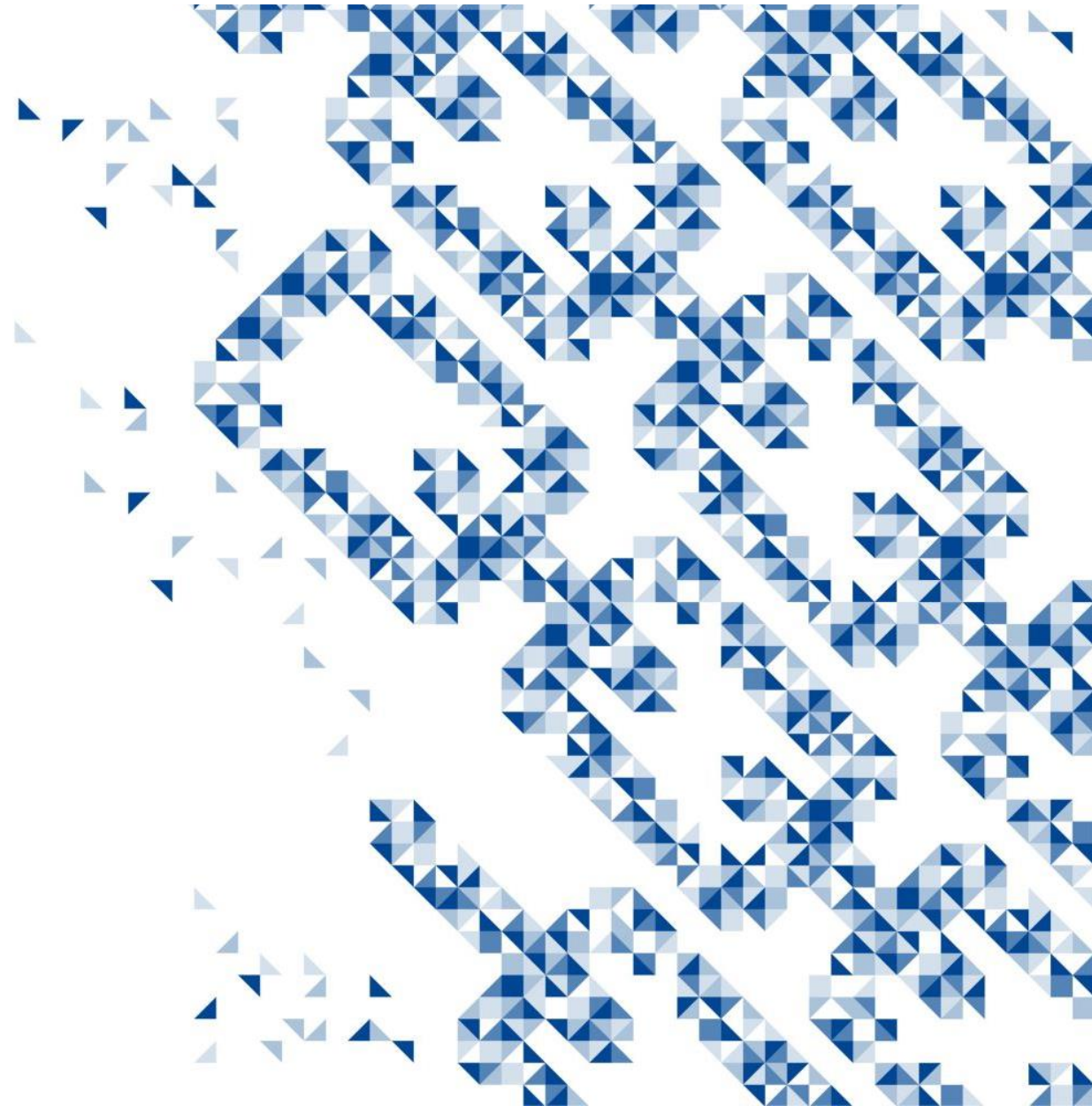
- Clustering Gene Segmentation Data
- Image Segmentation
- News Article Clustering
- Clustering Languages
- Species Clustering
- Anomaly Detection

Effective Use of K Means Clustering



Hierarchical Cluster

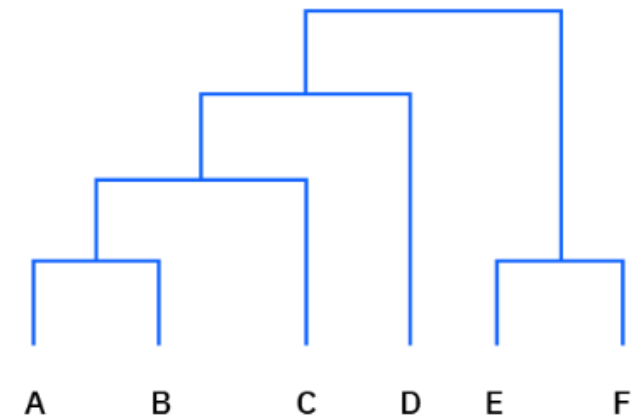
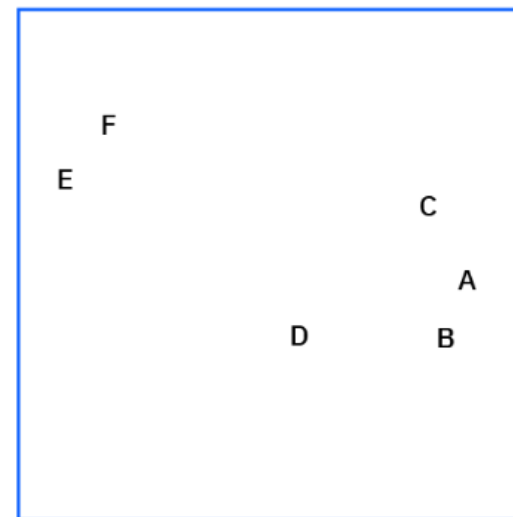
- **Hierarchical clustering**, also known as *hierarchical cluster analysis*, is an algorithm that groups similar objects into groups called **clusters**. The endpoint is a set of clusters, where each cluster is distinct from the others, and the objects within each cluster are broadly similar to one another.



There are two types of hierarchical clustering:

- Agglomerative or bottom-up approach that repeatedly merges clusters into larger ones until a single cluster emerges.
- Divisive or top-down approach that starts with all data in a single cluster and continues to split out successive clusters until all clusters are singletons.

A tree-like diagram called a dendrogram is often used to visualize the hierarchy of clusters. *It displays the order in which clusters have been merged or divided and shows the similarity or distance between data points.*



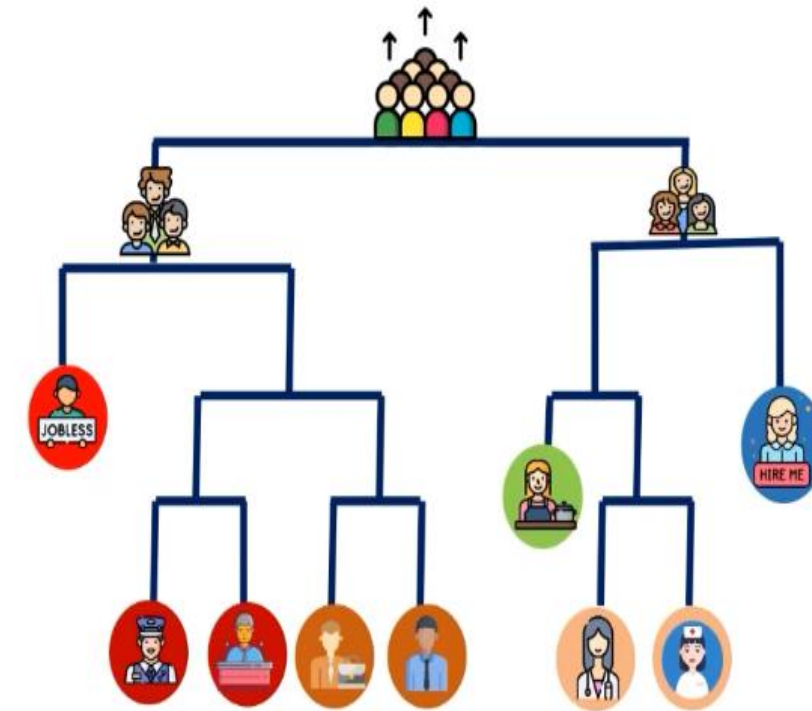
Agglomerative Algorithm

We start with each object in a cluster of its own and then repeatedly merge the closest pair of clusters until we end up with just one cluster containing everything.

The basic algorithm is as follows:

- Assign each object to its own single-object cluster and calculate the distance between each pair of clusters.
- Choose the closest pair of clusters and merge them into a single cluster (thus reducing the total number of clusters by one).
- Calculate the distance between the new cluster and each of the old clusters.
- Repeat steps 2 and 3 until all the objects are in a single cluster.

AG
G
L
O
M
E
R
A
T
I
V
E



Practical Ex:

<https://www.youtube.com/watch?v=faGhn73Qk44>

Steps

Step 1 — Start with individual clusters

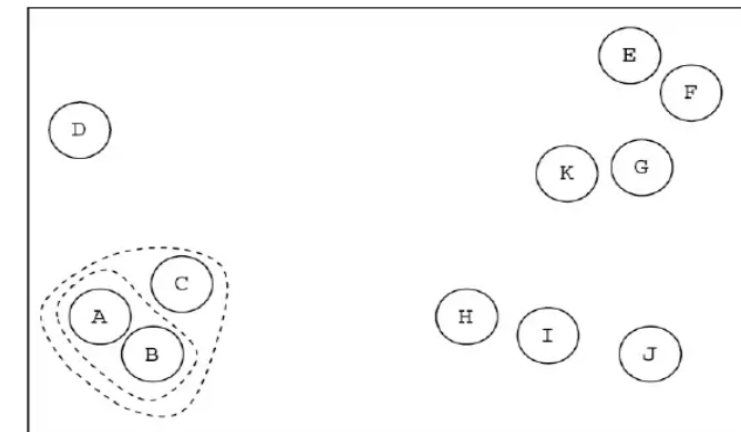
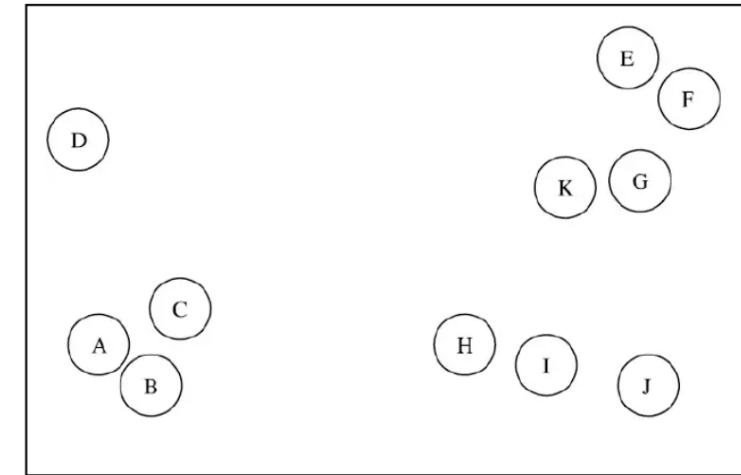
- Treat each object as its own cluster.
(For n objects, you start with n clusters.)

Step 2 — Compute all pairwise distances

- Calculate the distance between **every pair of clusters**.
- At the beginning, this is just the distance between individual points.

Step 3 — Merge the closest two clusters

- Identify the **two clusters** with the **smallest distance**.
- Merge them into a **new cluster**.
- Reduce the total number of clusters by one.



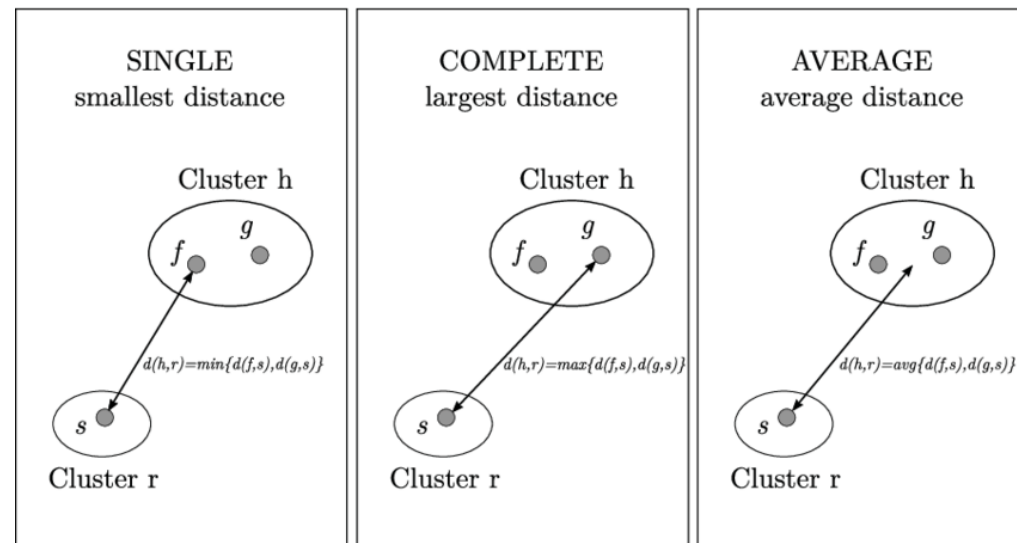
A and B $\rightarrow$ AB mean “clusters A and B are merged to form a new cluster, which we will call AB”

Steps

Step 4 — Recalculate distances

- After forming the new cluster:
- Compute the distance between this new cluster and **each of the remaining old clusters**.
- The way you compute this depends on the **linkage method**, such as:
 - *Single linkage* (nearest neighbor)
 - *Complete linkage* (farthest neighbor)
 - *Average linkage* (mean distance)

- A and B $\rightarrow$ AB
- AB and C $\rightarrow$ ABC
- G and K $\rightarrow$ GK
- E and F $\rightarrow$ EF
- H and I $\rightarrow$ HI
- EF and GK $\rightarrow$ EFGK
- HI and J $\rightarrow$ HIJ
- ABC and D $\rightarrow$ ABCD
- EFGK and HIJ $\rightarrow$ EFGKHIJ
- ABCD and EFGKHIJ $\rightarrow$ ABCDEFGKHIJ



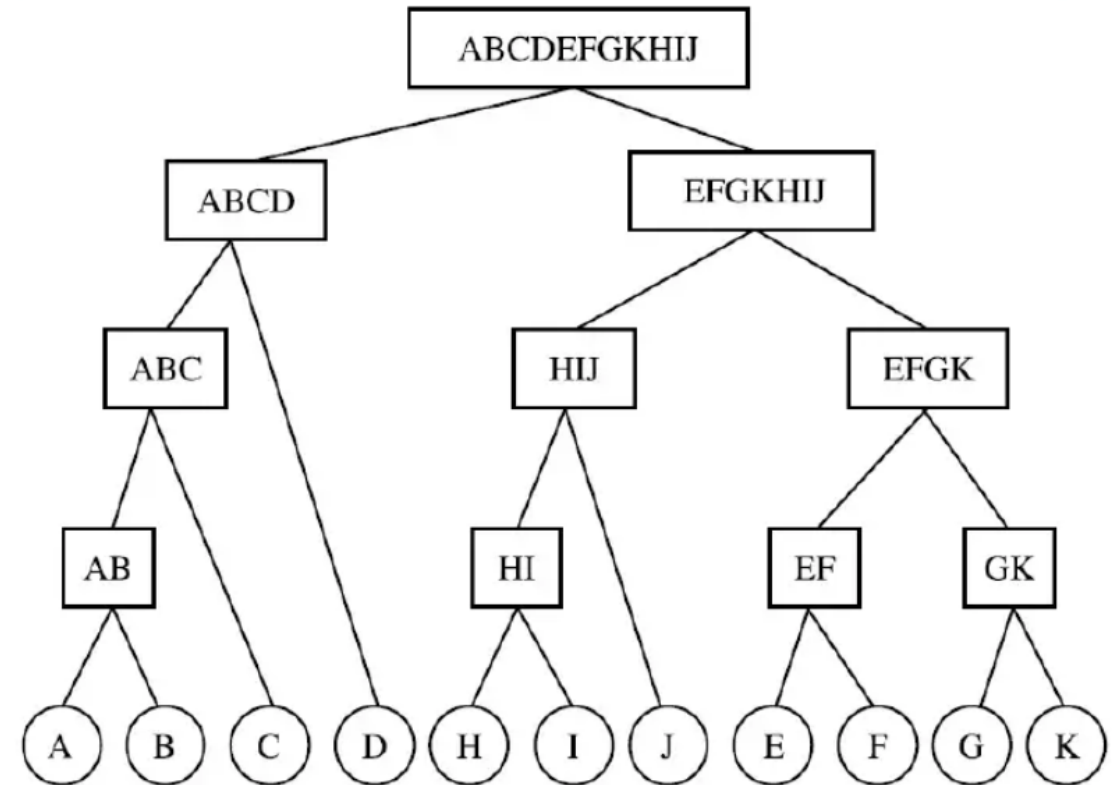
Steps

Step 5 — Repeat until only one cluster remains

- Continue repeating Steps 3 and 4:
 - Find the closest pair
 - Merge
 - Recalculate distances
- Stop when all objects are grouped into **one final cluster**.

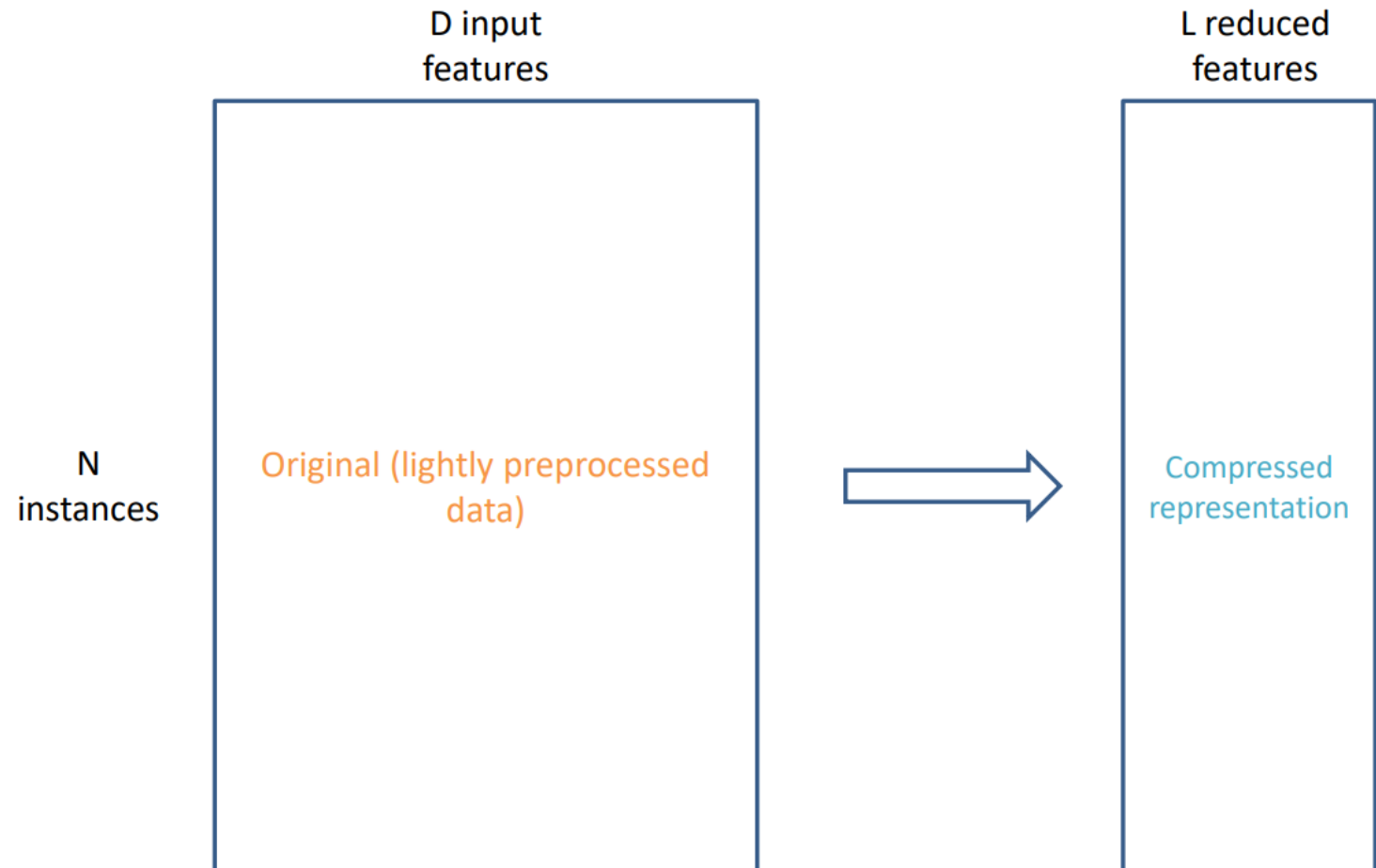
Step 6 — Draw the dendrogram

- Use the merge history to build a **dendrogram** (binary tree).
- All original objects appear at the bottom as **leaf nodes**.
- Branch height represents **distance at which clusters were merged**.

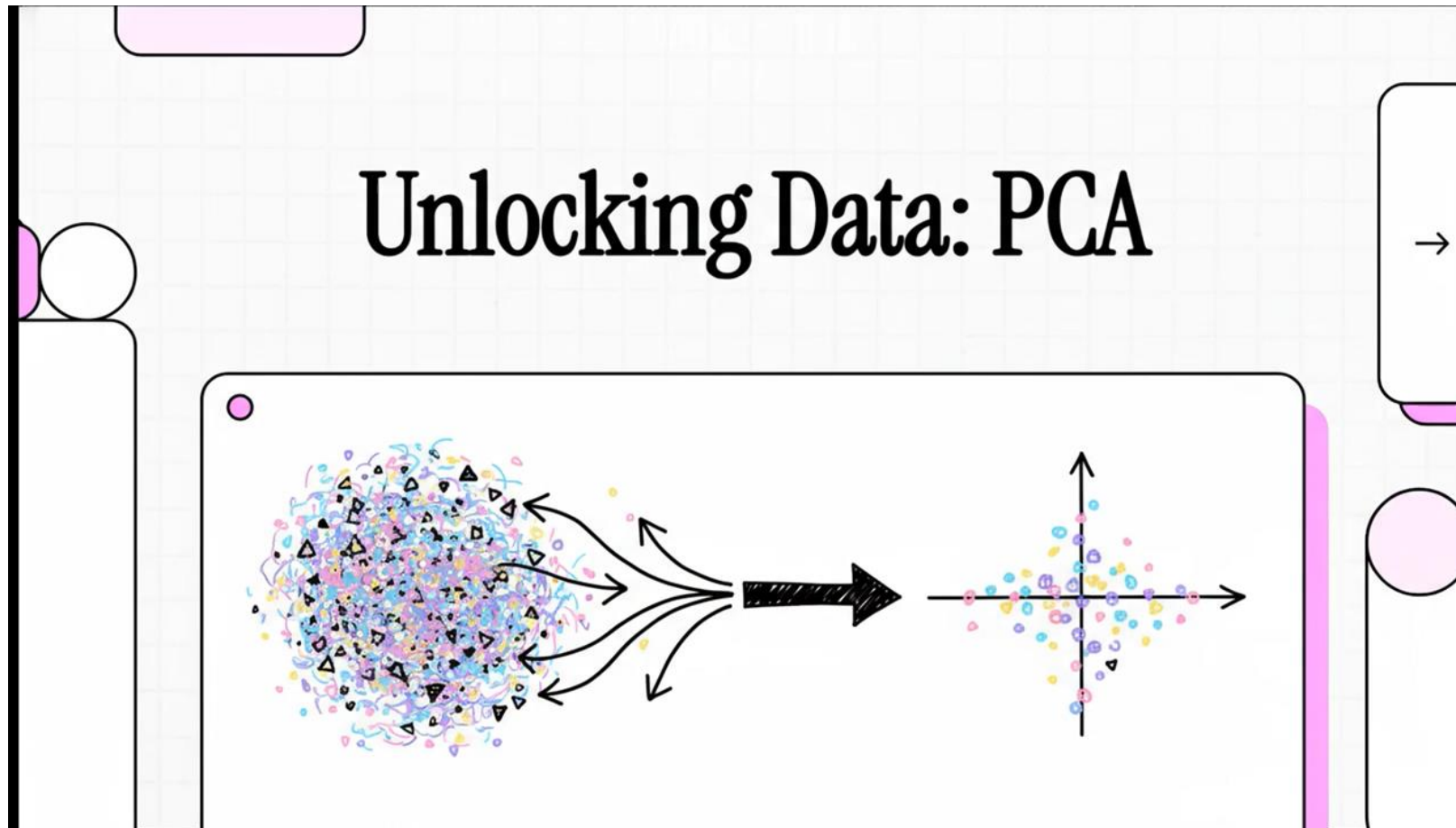


Dimension Reduction

Dimensionality Reduction



Principal component analysis (PCA)



Principal Component Analysis (PCA)



Principal Component Analysis is a well-known dimension reduction technique.



It transforms the variables into a new set of variables called as principal components.



These principal components are linear combination of original variables and are orthogonal.



The first principal component accounts for most of the possible variation of original data.



The second principal component does its best to capture the variance in the data.

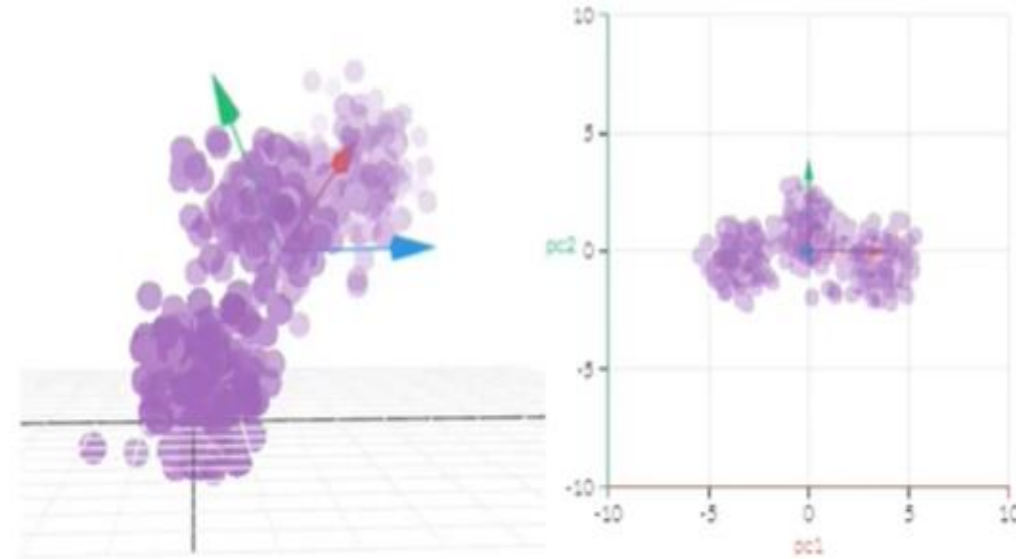


There can be only two principal components for a two-dimensional data set.

- **PCA** is used to summarize or transform highly correlated features of data into a few main, **uncorrelated components**.
- The PCA algorithm **orders the eigenvectors from highest to lowest** according to their **usefulness** in explaining the **total variance** in the initial data (i.e., eigenvalues).
- The **PC1** of a set of features $X_1, X_2, \dots, X_p \rightarrow$ is the **normalized linear combination** of the features that has **the largest variance**.

$$PC_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p \text{ where } \sum_{j=1}^p \phi_{j1}^2 = 1$$

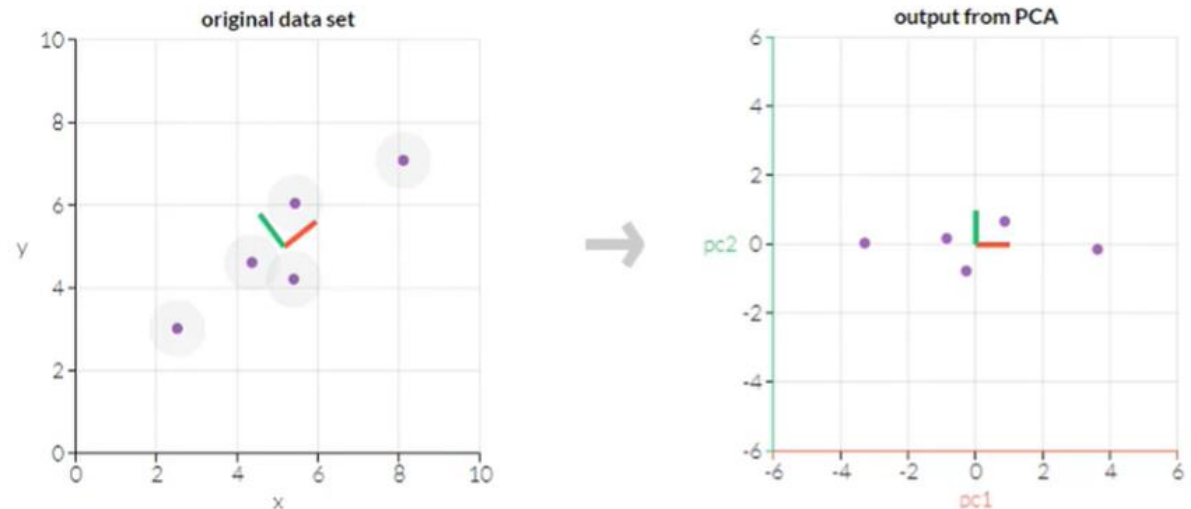
- The elements $\phi_{11}, \dots, \phi_{p1}$ are referred to as **loadings** of PC1.
- Note that the X features are **standardized (why?)**.
- The loading vector ϕ_1 defines a direction in feature space along which the data vary the most, i.e., maximizing the variance in that direction!



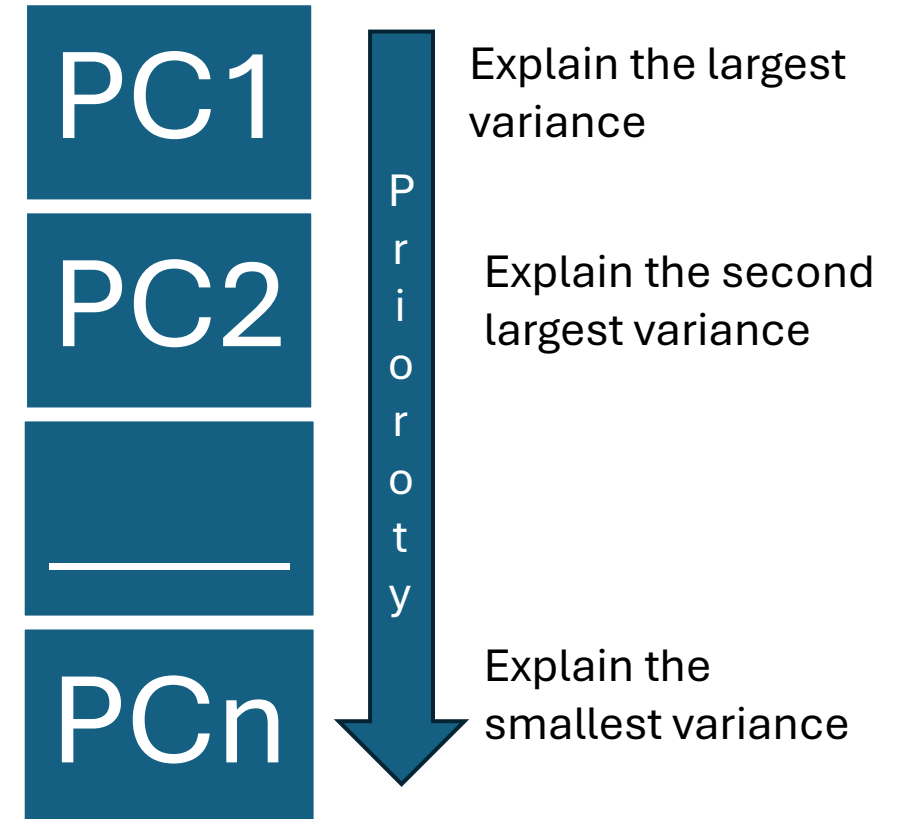
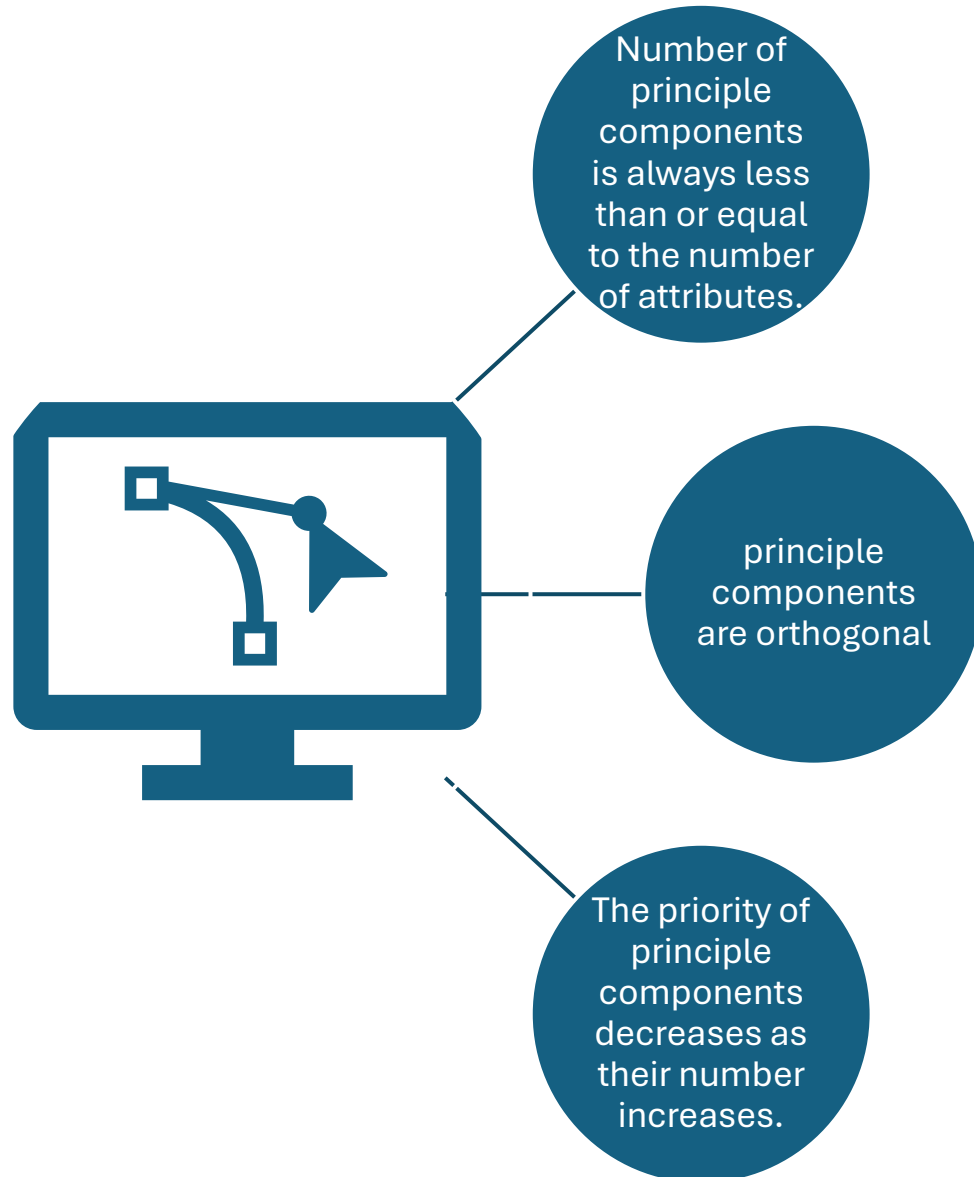
- The goal of PCA is to identify the **directions in the dataset along which the data vary the most**. These directions capture the maximum amount of information (variance) present in the data.
- To achieve this, PCA analyzes the **covariance matrix** of the dataset. The covariance matrix describes how variables vary together.
 - The **eigenvectors** of the covariance matrix represent the **principal components**. These define the **new directions (axes)** in the transformed feature space.
 - The **eigenvalues** corresponding to those eigenvectors represent the **amount of variance explained** along each principal component.

In simpler terms:

- **Eigenvectors** → **directions of maximum variation**
- **Eigenvalues** → **how much variation is captured in each direction**



The first principal component captures the greatest variance, the second captures the next highest variance (while being orthogonal to the first), and so on.



Covariance Matrix Computation

Covariance matrix is used to express the correlation between any two or more attributes in a multidimensional dataset.

The covariance matrix has the entries as the variance and covariance of the attribute values. The variance is denoted by “Var” and covariance is denoted by “Cov”.

Ex: The covariance matrix for two attributes and their values.

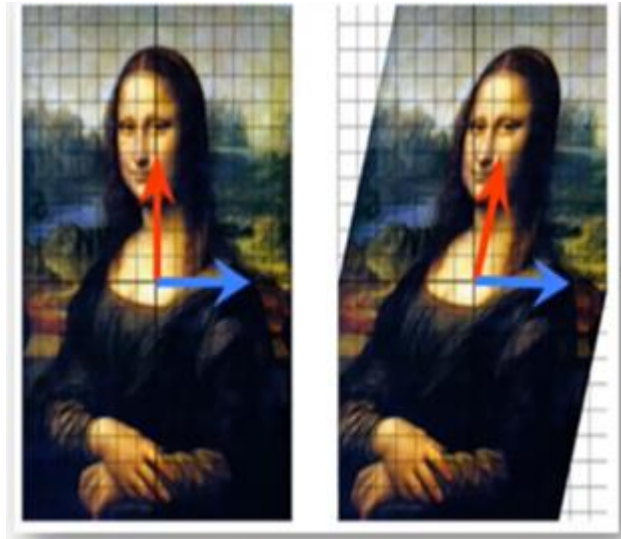
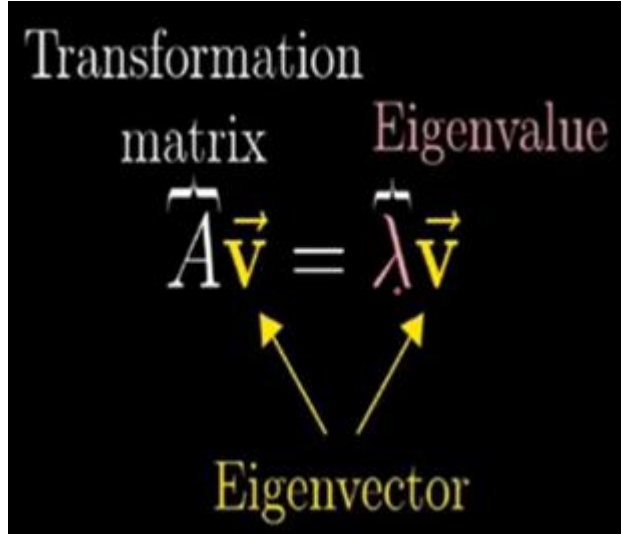


$$\begin{bmatrix} \text{Var}(x) & \text{Cov}(x, y) \\ \text{Cov}(x, y) & \text{Var}(y) \end{bmatrix}$$

Covariance matrix tells us how the two or more variables are related.

- **Positive covariance** indicate that the value of one variable is directly proportional to other variable.
- **Negative covariance** indicate that the value of one variable is inversely proportional to other variable.

$$\begin{bmatrix} \text{Var}(x) & \text{Cov}(x,y) & \dots & \text{Cov}(x,m) \\ \text{Cov}(x,y) & & & \\ \text{Cov}(z,x) & & & \\ \vdots & & & \\ \text{Cov}(n,x) & \dots & \dots & \text{Var}(n) \end{bmatrix}$$



Example: For this matrix

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix}$$

an eigenvector is

$$\begin{bmatrix} 1 \\ 4 \end{bmatrix}$$

with a matching eigenvalue of 6

Let's do some matrix multiplies to see if that is true.

Av gives us:

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -6 \times 1 + 3 \times 4 \\ 4 \times 1 + 5 \times 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

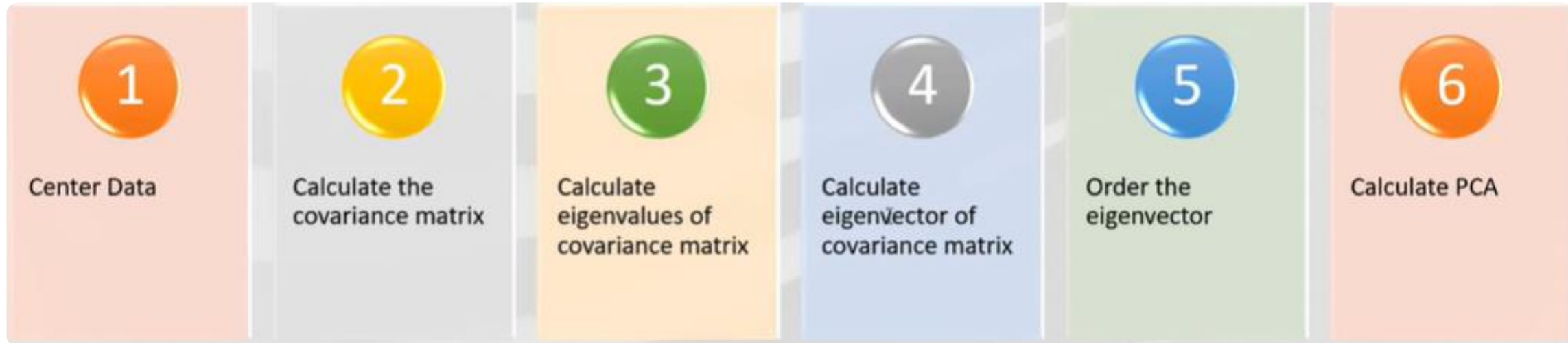
λv gives us :

$$6 \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$$

Yes they are equal!

So we get $Av = \lambda v$ as promised.

Steps to find PCA



STEP 1: STANDARDIZATION

Purpose:

Different variables often have different ranges.
If we do not standardize them:

- Variables with **larger scales** dominate
- Variables with **smaller ranges** contribute less
- The final PCA results become **biased**

Standardization ensures that **each variable contributes equally** to the analysis.

F1	F2	F3	F4
-1.0695	0.8196	0	-1
0.5347	-1.6393	1.6042	1
-1.0695	0	0	0
0.5347	0	-1.0695	-1
1.0695	0.8196	-0.5	1



Standardization Formula : $Z = \frac{X-\mu}{\sigma}$

Where:

- X = original value
- μ = mean of the variable
- σ = standard deviation
- Z = standardized value

Large Size Apples (F1)	Rotten Apples (F2)	Damaged Apples (F3)	Small Apples (F4)
1	5	3	1
4	2	6	3
1	4	3	2
4	4	1	1
5	5	2	3



	F1	F2	F3	F4
Mean	3	4	3	2
Std	1.87	1.223	1.87	1



STEP 2: COVARIANCE MATRIX COMPUTATION

Purpose

In this step, we determine **how the variables vary together** with respect to their mean values.

It helps us:

- Identify relationships between variables
- Detect highly correlated features
- Understand the structure of the dataset

It's a symmetric matrix (N×N) that contains:

- Variances on the diagonal
- Covariances off the diagonal



Important Properties

1. $Cov(X, X) = Var(X)$
2. $Cov(X, Y) = Cov(Y, X) \rightarrow$ symmetric
3. Positive value $\rightarrow$ direct correlation
4. Negative value $\rightarrow$ inverse correlation
5. Zero $\rightarrow$ no linear relationship

Covariance Formula

For two variables X and Y :

$$Cov(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})$$

Where:

- X_i, Y_i = data values
- $\bar{X}, \bar{Y}$ = means
- n = number of observations

Since the data is already standardized: $\bar{X} = 0$

So, the formula simplifies to:

$$Cov(X, Y) = \frac{1}{n-1} \sum X_i Y_i$$

Covariance Matrix for Two-Dimensional Data

For variables X and Y :

$$\text{Covariance Matrix} = \begin{bmatrix} Var(X) & Cov(X, Y) \\ Cov(Y, X) & Var(Y) \end{bmatrix}$$

	F1	F2	F3	F4
F1	VAR(F1)	COV(F1, F2)	COV(F1, F3)	COV(F1, F4)
F2	COV(F2, F1)	VAR(F2)	COV(F2, F3)	COV(F2, F4)
F3	COV(F3, F1)	COV(F3, F2)	VAR(F3)	COV(F3, F4)
F4	COV(F4, F1)	COV(F4, F2)	COV(F4, F3)	VAR(F4)

Standardized values

F1	F2	F3	F4
-1.0695	0.8196	0	-1
0.5347	-1.6393	1.6042	1
-1.0695	0	0	0
0.5347	0	-1.0695	-1
1.0695	0.8196	-0.5	1

Variance of F1 : $Var(F1) = \frac{1}{n-1} \sum (F1_i - 0)^2$

$$\begin{aligned}
 Var(F1) &= \frac{(-1.0695)^2 + (0.5347)^2 + (-1.0695)^2 + (0.5347)^2 + (1.0695)^2}{5} \\
 &= \frac{1.144 + 0.286 + 1.144 + 0.286 + 1.144}{5} \\
 &= \frac{4.004}{5} = 0.78
 \end{aligned}$$

- Variance of F1 = 0.78 → moderate spread
- Covariance(F1,F2) = -0.8586 → strong negative relationship
→ When F1 increases, F2 tends to decrease

Covariance Between F1 and F2: $Cov(F1,F2) = \frac{1}{n-1} \sum (F1_i)(F2_i)$

Substitute standardized values:

$$\begin{aligned}
 &= \frac{(-1.0695)(0.8196) + (0.5347)(-1.6393) + (-1.0695)(0) + (0.5347)(0) + (1.0695)(0.8196)}{5} \\
 &= -0.8586
 \end{aligned}$$

Similarly solving all the features, the covariance matrix will be,

- The matrix is symmetric.
- Diagonal values (0.78) are variances.
- Off-diagonal values are covariances.
- Negative values indicate inverse relationships.
- Positive values indicate direct relationships.



	F1	F2	F3	F4
F1	0.78	-0.8586	-0.055	0.424
F2	-0.8586	0.78	-0.607	-0.326
F3	-0.055	-0.607	0.78	0.426
F4	0.424	-0.326	0.426	0.78

STEP 3 & 4: FEATURE VECTOR (Eigenvalues & Eigenvectors)

Purpose

In this step, we find the **principal components**.

Mathematically: $Av = \lambda v$

Where:

- A = covariance matrix
- v = eigenvector
- λ = eigenvalue

To find the Eigenvalues, we solve:

$$\det(A - \lambda I) = 0$$

This is called the **characteristic equation**.

- It expands into a polynomial equation in λ .
- Solving it gives eigenvalues.



Why Eigenvalues Matter?

Eigenvalues = strength of each principal component

- Each eigenvalue represents the **variance captured** by its eigenvector.
- The eigenvector corresponding to the largest eigenvalue becomes:

First *Principal Component* (PC1)



Why Eigenvectors represent?

Each eigenvector:

- Defines a **new direction (axis)** in the data space.
- Is a weighted combination of F1, F2, F3, F4.
- Shows how original features combine to form a principal component.

Example structure:

$$v_1 = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}$$

Meaning:

$$PC1 = w_1F1 + w_2F2 + w_3F3 + w_4F4$$

Why This Step Is Called "Feature Vector"?

After computing eigenvectors:

We build a matrix:

$$\text{FeatureVector} = [v_1 \ v_2 \ v_3 \ v_4]$$

Each column = one eigenvector.

This matrix will be used to rotate the dataset.



Concept	Meaning
Covariance Matrix	Relationship between features
Eigenvalue	Amount of variance explained
Eigenvector	Direction of maximum variance
Principal Component	Eigenvector ordered by eigenvalue

	F1	F2	F3	F4
F1	$0.78 - \lambda$	-0.8586	-0.055	0.424
F2	-0.8586	$0.78 - \lambda$	-0.607	-0.326
F3	-0.055	-0.607	$0.78 - \lambda$	0.426
F4	0.424	-0.326	0.426	$0.78 - \lambda$

$$A - \lambda I =$$

- **characteristic equation** $\det(A - \lambda I) = 0$

Expanding a 4×4 determinant produces a **4th-degree polynomial**:

$$\lambda^4 + a\lambda^3 + b\lambda^2 + c\lambda + d = 0$$

When you expand and simplify (very long algebra), you obtain:

$$(\lambda - 2.11691)(\lambda - 0.855413)(\lambda - 0.481689)(\lambda - 0.334007) = 0$$

Solving gives:

$$\lambda = 2.11691, 0.855413, 0.481689, 0.334007$$

- **Solve** $(A - \lambda I)v = 0$

For each eigenvalue:

1. Substitute λ into the matrix
2. Solve the linear system
3. Find a non-zero vector v

For $\lambda = 2.11691$

We solve:

$$(A - 2.11691I)v = 0$$

After solving the system (using elimination or Cramer's rule), we obtain:

$$v_1 = \begin{bmatrix} 0.515514 \\ -0.616625 \\ 0.399314 \\ 0.441098 \end{bmatrix}$$

This is the **first principal component direction (PC1)**.

It corresponds to the largest eigenvalue → explains most variance.

In reality we compute eigenvalues using:

Python:

```
import numpy as np
np.linalg.eig(A)
```

Repeat for Remaining Eigenvalues

Doing the same for:

$$\lambda_2 = 0.855413, \lambda_3 = 0.481689, \lambda_4 = 0.334007$$

gives eigenvectors:

$$v_2 = \begin{bmatrix} -0.627 \\ -0.537 \\ -0.455 \\ 0.340 \end{bmatrix}, v_3 = \begin{bmatrix} -0.229 \\ 0.558 \\ -0.785 \\ 0.134 \end{bmatrix}, v_4 = \begin{bmatrix} 0.538 \\ -0.144 \\ -0.122 \\ -0.820 \end{bmatrix}$$

- Eigenvalues tell you **how much variance**
- Eigenvectors tell you **in what direction**

STEP 5: RECAST THE DATA ALONG THE PRINCIPAL COMPONENTS AXES

Purpose

express the data in the **new coordinate system** defined by the principal components.

- 
- Old axes → F1, F2, F3, F4
 - New axes → PC1, PC2, PC3, PC4

Each column = one principal component.

	E1	E2	E3	E4
F1	0.515514	-0.623012	0.0349815	-0.587262
F2	-0.616625	0.113105	0.452326	-0.634336
F3	0.399314	0.744256	-0.280906	-0.455767
F4	0.441098	0.212477	0.845736	0.212173

PC1PC2PC3PC4

Final Data Set=Standardized Data×Feature Vector
Mathematically:

$$Z = X_{std} \cdot V$$

Where:

- 
- X_{std} =standardized dataset (5×4 matrix)
 - V = eigenvector matrix (4×4 matrix)
 - Z = transformed dataset (5×4 matrix)

Standardized Original Data Set

F1	F2	F3	F4
-1.0695	0.8196	0	-1
0.5347	-1.6393	1.6042	1
-1.0695	0	0	0
0.5347	0	-1.0695	-1
1.0695	0.8196	-0.5	1

FeatureVector

E1	E2
0.515514	-0.623012
-0.616625	0.113105
0.399314	0.744256
0.441098	0.212477

$$Z = X_{std} \cdot V$$

$$Z_{reduced} = X_{std} \cdot [E1 \ E2]$$

- Original 4D → becomes 2D
- Maximum variance preserved
- Minimal information loss

The projected dataset after PCA reduction to 2 principal components.

Large Size Apples	Rotten Apples
0.4268066978	0.00116114000000012
-0.4234920968	-0.39182856
-0.551342223	-0.7959219
0.4652040128	0.89996329
0.08272794800000002	0.28653037



The new 5×2 matrix

The percentage of variance

The eigenvalues:

$$\lambda = 2.11691, 0.855413, 0.481689, 0.334007$$

Total variance = sum of all eigenvalues

$$\begin{aligned}\text{Total} &= 2.11691 + 0.855413 + 0.481689 + 0.334007 \\ &= 3.788019\end{aligned}$$

Percentage Explained by Each Component

Formula:

$$\text{Explained Variance Ratio} = \frac{\lambda_i}{\text{Total Variance}} \times 100$$


◆ **PC1**

$$\frac{2.11691}{3.788019} \times 100 = 55.9\%$$

◆ **PC2**

$$\frac{0.855413}{3.788019} \times 100 = 22.6\%$$

◆ **PC3**

$$\frac{0.481689}{3.788019} \times 100 = 12.7\%$$

◆ **PC4**

$$\frac{0.334007}{3.788019} \times 100 = 8.8\%$$

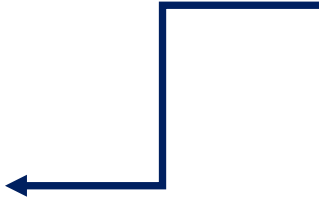
If You Keep Only 2 Components

$$55.9\% + 22.6\% = 78.5\%$$

So, by reducing from **4 features** → **2 principal components**,
you retain:

78.5% of the total variance

That means you kept almost 80% of the information.



Association Rule

What is Market–Basket Analysis?

- A technique used by large retailers to identify associations among items.
- It works by looking for combinations of items that occur frequently in transactions.
- It uses “Association Rules” for analyzing combinations of frequent item sets.

Association Rules

Let's Understand Association Rule...

- It's a technique to know how items are associated to each other.
- **Factors to be considered to measure association:**
 - Support: It states how popular an itemset is. It is measured by the proportion of transactions in which the itemset appears.
 - Confidence: It states how likely an item Y is purchased when item X is purchased ($X \rightarrow Y$). It is measured by the proportion of transactions containing item X in which item Y also appears.
 - Lift: It states how likely an item Y is purchased when item X is purchased, while controlling for how popular item Y is.

Example Transactions

Transaction	Items
1	   
2	  
3	 
4	 
5	   
6	  
7	 
8	 

Total number of transactions = 8

Support Calculation

Support(🍎)

🍎 appears in Transactions: **1, 2, 3, 4 → 4 times**

$$\text{Support}(\{\text{🍎}\}) = \frac{4}{8} = 0.5$$

Support(🍷)

🍷 appears in Transactions:

1, 2, 3, 5, 6, 7 → **6 times** $\text{Support}(\{\text{🍷}\}) = \frac{6}{8} = 0.75$

Support(🍎, 🍷)

Transactions that contain **both** 🍎 and 🍷:

Transactions **1, 2, 3 → 3 times**

$$\text{Support}(\{\text{🍎}, \text{🍷}\}) = \frac{3}{8} = 0.375$$

Confidence Calculation

Confidence (🍎 → 🍷)

$$\begin{aligned} \text{Confidence}(\{\text{🍎} \rightarrow \text{🍷}\}) &= \frac{\text{Support}(\{\text{🍎}, \text{🍷}\})}{\text{Support}(\{\text{🍎}\})} \\ &= \frac{0.375}{0.5} = 0.75 \end{aligned}$$

When customers buy 🍎, 75% of the time they also buy 🍷.

Lift Calculation

Lift (🍎 → 🍷)

$$\begin{aligned} \text{Lift}(\{\text{🍎} \rightarrow \text{🍷}\}) &= \frac{0.375}{0.5 \times 0.75} \\ &= \frac{0.375}{0.375} = 1 \end{aligned}$$

🍎 and 🍷 occur **together exactly as expected by chance**.
There is **no strong association** (Lift = 1).

- Association rule learning can be divided into three types of algorithms:



Apriori Algorithm

Apriori Algorithm is a basic method used in data analysis to find groups of items that often appear together in large sets of data.

STEP 1 — Set Minimum Support

- Choose a **minimum support threshold** (e.g., 30%).
- Support = frequency of itemset in all transactions.



STEP 2 — Find All Frequent 1-Itemsets (L1)

- Count how many times each item appears.
 - Keep only items whose support $\geq$ minimum support.
- These are your **Frequent 1-itemsets (L1)**.



STEP 3 — Generate Candidate 2-Itemsets (C2)

Combine L1 with itself to generate **all possible 2-item combinations**.

Example: If $L1 = \{A, B, C\}$, then $C2 = \{AB, AC, BC\}$.



STEP 4 — Prune Infrequent Candidates

For every candidate k-itemset:

- Count its support
- Remove (prune) those with support $<$ minimum support

Remaining sets = **Frequent 2-itemsets (L2)**



STEP 5 — Generate Higher-Order Itemsets (k+1)

Use L_k to generate $C(k+1)$:

- Join L_k with L_k to get larger candidate sets
- Example: If $L2$ contains $\{AB, AC\}$, possible $L3$ candidate = ABC
- Prune any candidate whose subsets are not frequent
→ This is the Apriori property at work.

Apriori Algorithm

STEP 6 — Count Support & Prune Again

- Count support for all candidates
 - Keep only those meeting minimum support
 - These become **L3, L4, L5...**
- Stop when no more frequent itemsets can be formed.



STEP 7 — Generate Association Rules

From each frequent itemset:

- Generate rules of form **X → Y**
- Compute:
 - **Confidence**
 - **Lift**
 - (Optionally) **Conviction, Leverage**

Keep rules meeting your thresholds (e.g., Confidence $\geq$ 70%, Lift $\geq$ 1).

- The algorithm makes rules to show how items are related.
- It checks these rules using support, confidence and lift to find the strongest ones.

Association Rule Applications: It has various applications in machine learning and data mining. Below are some popular applications of association rule learning:



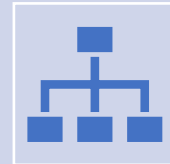
Market Basket Analysis: One of the most common applications of association rule mining, used by large retailers to determine relationships between items purchased together.



Medical Diagnosis: Association rules help identify the probability of certain diseases, making it easier to diagnose and treat patients.



Protein Sequence: Association rules assist in determining the synthesis of artificial proteins.



Catalog Design and Loss-leader Analysis: Association rules are used to optimize catalog layouts and identify products that attract customers, along with many other applications.

THANK
you